Stacked n = 83 K600 producer: probe pipeline + 16-byte slots + state fold + rank stage, end-to-end cold

2026-10-16

Stacked n = 83 K600 producer: probe pipeline + 16-byte slots + state fold + rank stage, end-to-end cold

Status (2026-10-16): protocol frozen; Tier 1 run in flight; no result. The frozen protocol and schedule were committed before any Tier 1 timing. The workflow `.github/workflows/n83-stacked.yml` runs them on one GitHub `ubuntu-24.04` runner. Every number in this README is a local diagnostic from the cloud-agent VM (a shared 4-vCPU Intel Xeon guest); none is evidence of a speedup.

Question

Main has five promoted n = 83 a = 1 K600 one-target IC improvements, each measured on its own parent and none end to end:

step	experiment	promoted on
lift_scan canonicalizer	`koblitz-n83-liftscan-20261015` (#926)	online, over B
Feistel scan order	`koblitz-n83-scanorder-20261015` (#927)	online, over B
both combined	`koblitz-n83-combined-20261015` (#929)	not promoted (`not_separated` on geomeans)
prefetched probes (pP)	`koblitz-n83-probepipe-20261015` (#931)	online 1.40x over the combined producer
two-level root table	`koblitz-n83-tablefilter-20261015` (#934)	online 1.063x over pP, almost all from the 16-byte slot (pK 1.058x)
folded S3 root index	`koblitz-n83-state-fold-20261015` (#933)	cold (setup + online) 1.84x over the combined producer; online about 5% slower; its rank stage on lift + Feistel was 2.25x cheaper per attempt (secondary)

None of them moved the n = 83 row of `docs/ic/BASELINES.md`, because none measured an end-to-end cold `S` (setup + index build + rank stage + online), and the rank stage (B's: about 750 s on the runner) dominates that cost.

Does the stack of all of them, with the rank stage run on the stack's own probe loop, lower the **end-to-end cold** cost of the current best producer (pP, whose rank stage is B's) on new targets, with every relation, scalar and factor-log table unchanged? That is an iteration under AGENTS.md "Success criterion"; a promoted result is the first n = 83 row with a measured end-to-end cold `S` after B.

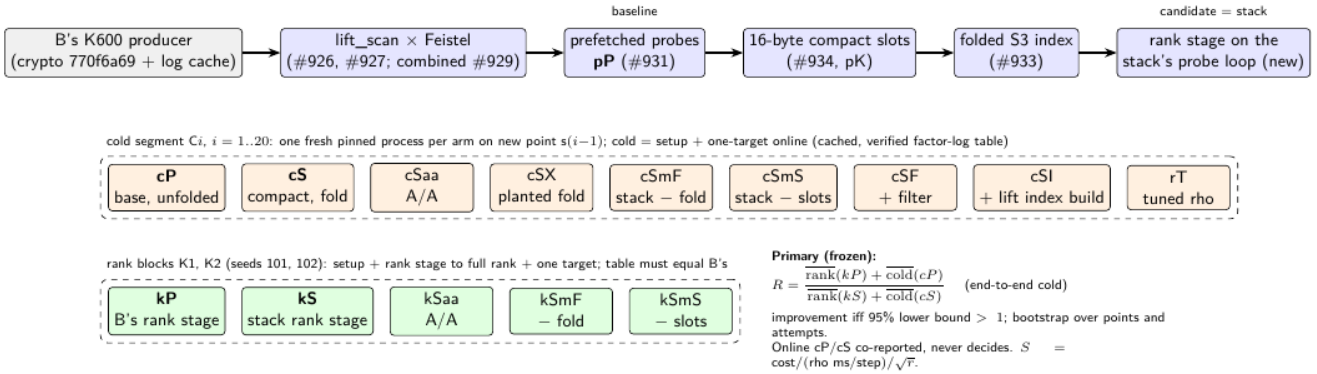


Figure 1: Design (from the frozen protocol; not a result). Source: figures/design.tex, vector: figures/design.pdf.

What is stacked

`make_online_source.py` derives the producer in two steps:

1. `make_tablefilter_source.py` is `koblitz-n83-tablefilter-20261015`'s `make_online_source.py`, verbatim (with its two verbatim parents, `make_probepipe_source.py` and `make_combined_source.py`). Its output must hash to that protocol's frozen producer digest (`70233510...`). With `KIC_ROOT_TABLE=base` and `KIC_PROBE_PIPELINE=prefetch` it is the probe-pipeline producer pP; with `KIC_ROOT_TABLE=compact` it runs the 16-byte slots.
2. Counted, exact replacements port the state fold (`koblitz-n83-state-fold-20261015`, `FOLD_CODE` and `fold_edits`) onto every root-table layout and probe loop:
   - `KIC_STATE_FOLD=1` stores 14,940,300 representatives of the 29,880,000 ordered states, fills **the selected table** (32-byte or 16-byte slots, or the filter's compact table) from them in the baseline's insertion order (capacity follows the stored count, 2^26 slots), and the folded scan now calls the layout's own probe loop (`scan_state128_select::<C, P>`) for each class at its first occurrence in the baseline order; mates are rebuilt by two rotations. `KIC_FOLD_PLANT` and `KIC_INDEX_CANON` keep the state fold's semantics.
   - **The rank stage** gets `KIC_RANK_CANON` and `KIC_RANK_ORDER` (as in the state fold) and the new `KIC_RANK_PROBE=target`, which runs the arm's own probe loop in the rank stage: the prefetch pipeline on the 32-byte table, or the compact (or filter) loop on the 16-byte table. So the stack's rank stage runs on the 16-byte table in the same process; the parent's rule that a non-base layout needs the cached factor-log table now applies only to `KIC_RANK_PROBE=off`. Each rank process records every attempt's probes and ns and `rank_relations_blake3`, a digest of every attempt's relation.
   - `KIC_INDEX_DIGEST=1` writes the state fold's index digests (logical states in index order; the table's key -> value map), read from whichever table the index holds, after the target and outside every interval.

Exactness argument. Each part is exact on its own (its parent's tests and gates): the 16-byte slot stores the same key -> value map and the same probe sequence; the pipeline keeps probe order and early return; the fold finds the baseline's first relation with fewer probes (the skipped mates made the same probes without a hit). The port changes only which loop the folded scan calls, and every loop takes a `State128`, so a folded scan over the compact table makes the compact loop's probes on the baseline's logical order. In the rank stage, the attempts are a deterministic function of the rank seed and the earlier relations; if every attempt's relation is unchanged, the rank stage's echelon, attempts and factor-log table are unchanged. The tests and the smoke below check all of this.

Left out, and why.

- **The root filter** (`KIC_ROOT_TABLE=filter`) is not in the stack: on the runner it added 1.005x on top of the 16-byte slot (run 38829449594). It is a secondary arm (`cSF`), cold only.
- **lift_scan in the index build** (`KIC_INDEX_CANON=lift`, the state fold's cF/cFI 1.33x cold secondary) was never promoted. It is a secondary arm (`cSI`), cold only; the time budget did not allow its own rank arm, and its rank stage is the stack's (it changes only the build).

Frozen Tier 1 design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, one fresh process per run. Every IC arm runs lift_scan, the Feistel order and the prefetch pipeline on the online path.

Arm	Table	Fold	Rank stage	Role
`cP` / `kP`	32-byte	no	B's: full scan, left-major, sequential probes	**baseline** pP
`cS` / `kS`	16-byte	yes	lift, Feistel, the stack's compact loop	**candidate**: the stack
`cSaa` / `kSaa`	16-byte	yes	as kS	A/A
`cSX`	16-byte	yes + planted	(cold only)	planted positive control: the fold's removed work, redone and discarded
`cSmF` / `kSmF`	16-byte	no	lift, Feistel, compact loop	ablation: stack minus fold
`cSmS` / `kSmS`	32-byte	yes	lift, Feistel, prefetch pipeline	ablation: stack minus slots
`cSF`	filter + 16-byte	yes	(cold only)	optional component: 32 MiB filter
`cSI`	16-byte	yes, lift in build	(cold only)	optional component: lift_scan index build
`rT`	—	—	—	tuned rho rT770 (Fold/64/DP6), seed 870000 + point seed

- **Cold panel.** 20 new public points (`inputs/cold_targets.json`, role `stacked-cold-v1`, `make_targets.py`), checked disjoint from Q0, B's panel and every earlier n = 83 panel of the law on main (scan-order held-out 384, combined 576, probe-pipeline 384, state-fold 16, table-filter 384), all regenerated. No scalar is known for any of them. Each `C<i>` segment runs one process per cold arm on point s(i-1), shuffled.
- **Rank blocks.** `K1`, `K2` (rank seeds 101, 102): one process per rank arm, each: setup, the rank stage to full rank, linear algebra, then one target (s000, s001). Every written factor-log table must equal B's archived table, log for log.
- **Q0 block** `O1`: cP and cS on Q0, exactness against the archived Feistel identity.
- **Segment order:** O1, C1-C4, K1, C5-C12, K2, C13-C20; 300-minute guard.

Primary (pre-registered): the end-to-end cold ratio. E2E(k, c) = mean rank stage of rank arm k over its two processes + mean cold (setup + one-target online, cached table) of cold arm c over the 20 points. R = E2E(kP, cP) / E2E(kS, cS); bootstrap: points resampled (paired), and the attempts of each rank process resampled (its non-attempt time kept), 10,000 draws. **Improvement** iff the 95% lower bound of R exceeds 1; regression iff the upper bound is below 1; otherwise not separated. **Invalid** if any gate or control fails or any scheduled row is missing.

Online is co-reported, never decides: online cP/cS (means and geomeans), and the fold's online cost inside the stack (online cSmF/cS).

- **Controls.** A/A: E2E(kS, cS)/E2E(kSaa, cSaa) and cold cS/cSaa both inside [0.97, 1.03]. Planted: cold cSX/cS lower bound above 1.
- **Gates.** Every IC process: its layout, slot count (2^27 unfolded, 2^26 folded), 29,880,000 logical states, 14,940,300 stored iff folded, 29,878,182 table entries, and B's index digests. Per point: one scalar across every IC process and rT (oracle replay); one relation across every IC process; probes cP = cSmF; cS = cSaa = cSX = cSmS = cSF = cSI $\leq$ cP; cSX + audit = cP. Rank: B's table; per block, one relation digest across kS, kSaa, kSmF, kSmS; identical per-attempt probes across kS, kSaa, kSmS; kSmF $\geq$ kS per attempt. Q0: archived relation and scalar, probes 1,988,184 (cP) and 1,987,852 (cS).
- **Secondaries.** Online, cold-without-rank, setup, index and peak-RSS ratios cP/cS; ablations E2E(kSmF, cSmF)/E2E(kS, cS) and E2E(kSmS, cSmS)/E2E(kS, cS) with their online, cold and setup ratios; cS over cSF and cSI; rank-stage and per-attempt ratios kP/kS, kSmF/kS, kSmS/kS; each rank process's single-process

end-to-end cold time; `S` = cost / (rT ms/step) / sqrt(r) for end-to-end cold, cold without rank and online, per arm; delta `S` cP to cS; the end-to-end cold gap IC / rT and the online gap to rT (distance remaining, not a gate).

- **Power** (`power_check.py`, `analysis/power.json`; prior `inputs/power_prior.json` built by `make_power_prior.py` from the state fold's run 38819494806 and three measured factors). The rank stage, not the point count, limits resolution. At the minimal effect of interest, 1.15, the power is 0.88-0.93 at every size from 4 to 20 points (300 simulations; the smallest size passing 0.80 after two Monte Carlo SEs is 4); at 1.10 it is 0.55-0.63, so end-to-end effects of 10% or less are not reliably resolved by this design. The prior's expected ratio is 3.2 (pP rank stage about 757 s, cold about 42 s; stack about 228 s and 22 s). 20 points serve the per-point online ablations (power to see the fold's 5% online cost inside the stack: 0.98 at 8 points, 1.0 at 16-20).
- **Runtime estimate.** About 3 h including the build (up to about 3.5 h on a slower runner); 300-minute time guard, 350-minute job limit.

Local validation (cloud-agent VM; diagnostics, no timing claims)

See `smoke/README.md`.

Files

File	Role
`protocol.json`, `make_protocol.py`, `schedule.json`, `make_schedule.py`	frozen protocol, its builder, frozen run order
`make_online_source.py`, `make_tablefilter_source.py`, `make_probepipe_source.py`, `make_combined_source.py`, `online/`, `lift2/`	the stacked producer (verbatim parent derivations, digest-checked, plus the fold port and rank selectors); tuned rho; the two-limb canonicalizer crate
`restore.py`, `check_log_table.py`	restore and digest-check crypto 770f6a69 and every local source; re-verify B's factor-log table
`make_targets.py`, `inputs/cold_targets.json`	the new cold panel
`inputs/archived_identities.json`	archived Q0 identity (from `koblitz-n83-tablefilter-20261015`)
`make_power_prior.py`, `inputs/power_prior.json`, `power_check.py`, `analysis/power.json`	power prior, simulation, result
`arm.py`, `run_schedule.py`	one arm per pinned process; the schedule runner
`verify_receipt.py`	gates, statistics, decision rule; `result.json`, `result_table.md`
`figures/design.*`	design diagram (TikZ source, PDF, PNG)
`smoke/`	local diagnostics